

Stop sequences and controlling generation

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 8 of 12 · English notes

Session 8 (19 August 2026) opened by finishing a control the 17 August lab had already named: **stop sequences**. You pass one or more strings on the Messages request. When the model emits a string that matches, generation **halts**. The tokenizer still bills the tokens that were actually produced — including punctuation that pushed the count to 14 in the 1,2,3,4 demo.

What stop sequences do — and do not do

Vijay's slogan: the feature **ends content generation**; it does **not filter** a longer answer after the fact. The model is not writing 1–100 internally and then cropping to 5–10 for the client. Combined with **prefill** (assistant message that already contains the start of the answer), you are steering *where generation begins and ends*, not post-processing a hidden full essay.

“CLAUDE-ONLY” IN CLASS

The trainer presented stop sequences as a Claude-specific control versus “write a prompt that says start at 5 and stop at 10.” Other vendors also have stop parameters. For the exam, know Claude's `stop_sequences` field and `stop_reason`; do not wage a vendor war in the written paper. Confirm the live field name in the Messages API.

Prefill plus stop: the counting example

Mritunjay worried that the model “knows” 1–100 and that output tokens would still count the unused numbers. Vijay's correction: if you prefill so continuation starts at 5 and you stop at 10 (or at the token , 5 in the printed 1,2,3,4 example), the client only receives that span. Output tokens billed are that span, not a secret 1–100. Prefill + stop is a **prompting / decoding control**, not a second LLM pass that truncates.

He also limited the technique: it shines on **sequential / delimited** content (lists, fenced code, JSON braces). For messy reasoning or “do all the maths then give a short answer,” stop sequences are the wrong tool — use clearer instructions, interpolation, or (later) structured outputs / tools.

stop_reason you must recognise

Class language	Typical meaning	What you should do
Stop sequence	A configured string appeared in the output	Parse up to (usually excluding) the stop string; you got a controlled cut
EOS / end of content / end turn	Model finished naturally	Treat as complete unless the text is still semantically unfinished
Max tokens / token limit	Hit <code>max_tokens</code>	Retry with a larger cap, or design a shorter target format

- **Streaming**

<https://docs.claude.com/en/docs/build-with-claude/streaming>

How stops appear when tokens arrive as deltas.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.